

西安交通大学

课程实验报告

实验名称: 二次规划问题编程证明

姓名: 林圣翔

学号: 2223312202

专业班级: 计试 2201

指导教师: 罗敏楠教授

报告日期: 2025.10.13



一、实验目的

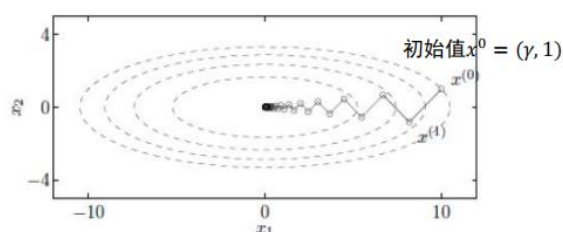
1. 研究条件数对梯度下降法收敛性能的影响。
2. 提升科学计算和结果可视化的实践能力。

二、实验内容

例1 两个变量的二次规划问题： $\min f(x) = \frac{1}{2}(x_1^2 + \gamma x_2^2)$ ， $\gamma > 0$ ，最优解 $x^* = (0,0)$ 。

$$x^k = \left(\frac{\gamma-1}{\gamma+1}\right)^k \begin{bmatrix} \gamma \\ (-1)^k \end{bmatrix}$$

- 其Hessian矩阵 $\nabla^2 f(x)$ 是常值，其特征值为1和 γ ，则 $\frac{M}{m} = \frac{\max\{1,\gamma\}}{\min\{1,\gamma\}} = \max\left\{\gamma, \frac{1}{\gamma}\right\}$
- $\gamma = 1$ 时，一次迭代可得到最优解；
- 对于 γ 离1不远的情况，如 $\frac{1}{3}$ 和3之间，收敛速度很快
- 如果 $\gamma \gg 1$ 或 $\gamma \ll 1$ 时，收敛速度将会非常慢



$\gamma = 10$ 时迭代过程

作业：编程证明以上结论

三、实验环境

1. 系统环境：基于 Ubuntu 20.04.6 LTS 版本
2. 编程语言：Python 3.10

四、实验过程

1. 环境配置

1.1. 创建 Conda 虚拟环境

```
conda create -n YHFFJC python=3.10
```

1.2. 激活 Conda 虚拟环境

```
conda activate YHFFJC
```

1.3. 安装项目依赖包

```
pip install -r requirements.txt
```

2. 代码实现

2.1. 具体函数及其功能如下表所示

函数名	功能描述
<code>analytical_solution(k, y)</code>	计算梯度下降法在二次优化问题中的解析解, 基于系数项 $(y-1)/(y+1)^k$ 分别计算 x_1 和 x_2 分量
<code>objective_function(x1, x2, y)</code>	计算二次目标函数值, 用于评估当前解的优劣
<code>iterations_to_convergence(y_values, tolerance_values, max_iterations)</code>	分析不同条件数和容忍度下梯度下降的收敛性能, 通过迭代计算解析解直至目标函数值低于容忍度阈值
<code>plot_iterations_to_convergence(y_values, tolerance_values)</code>	可视化收敛性能分析结果, 使用半对数坐标展示不同容忍度下迭代次数随条件数的变化趋势
<code>plot_3d_surface_and_trajectory()</code>	生成 3D 曲面和等高线图展示目标函数 <code>landscape</code> 及梯度下降轨迹, 直观呈现优化过程
<code>plot_initial_iterations()</code>	绘制前几次迭代中目标函数值的变化曲线, 展示不同条件数对收敛速度的影响

2.2. 主函数执行流程

2.2.1. 调用 `plot_initial_iterations()` 函数

取 $\gamma = [0.1, 0.5, 1, 2, 3, 5]$, 设置最大迭代次数为 10, 绘制前 10 次迭代过程中目标函数值的变化曲线, 展示不同条件数对梯度下降收敛速度的影响。

2.2.2. 调用 `plot_3d_surface_and_trajectory()` 函数

选取 $\gamma = [0.01, 0.1, 0.33, 1, 3, 10, 100]$ 代表性条件数, 分别生成目标函数的 3D 曲面图和等高线图, 并在图中绘制梯度下降的迭代轨迹, 直观呈现优化算法在不同地形条件下的搜索路径。

2.2.3. 调用 `plot_iterations_to_convergence(y_values, tolerance_values)` 函数

使用 $\gamma = [0.001, 0.01, 0.1, 0.33, 1, 3, 10, 100, 10000, 100000]$ 和容忍度 $\text{tolerance_values} = [1e-4, 1e-5, 1e-6, 1e-7, 1e-8]$, 系统分析不同精度要求下梯度下降达到收敛所需的迭代次数, 绘制收敛性能曲线并保存数值结果到 CSV 文件。

3. 运行代码

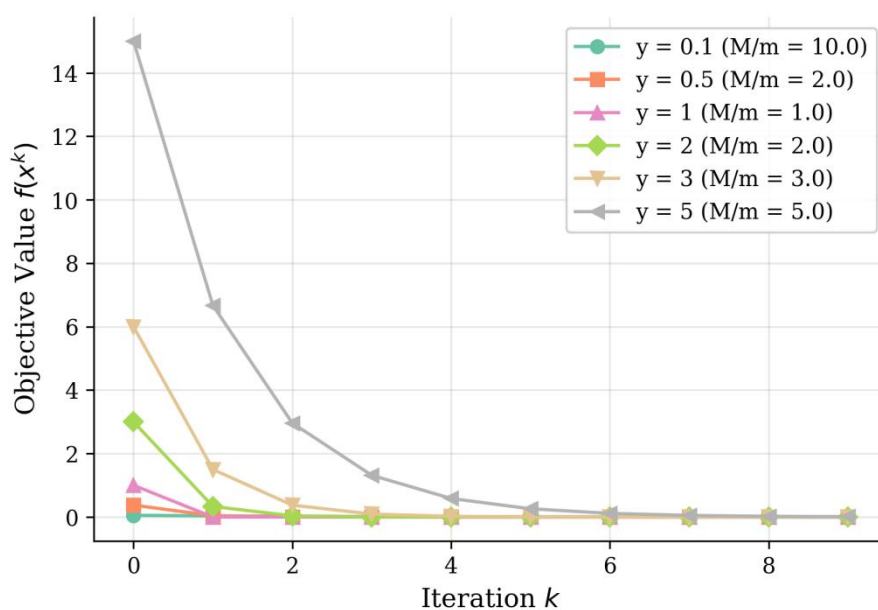
执行如下命令, 程序依次完成步骤 2.2.1-2.2.3。

```
python run.py
```

五、实验结果

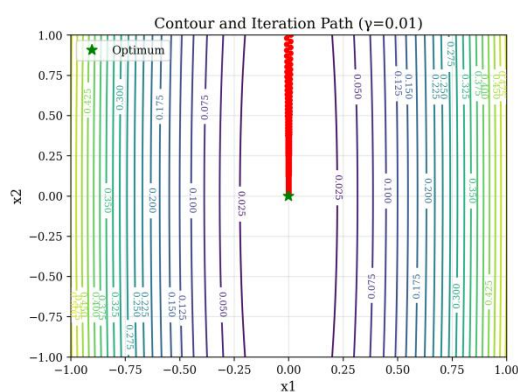
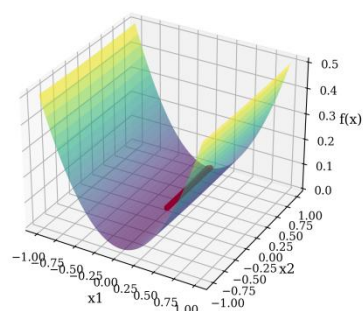
1. 不同条件数下的初期迭代情况

First few iterations for Different Condition Numbers

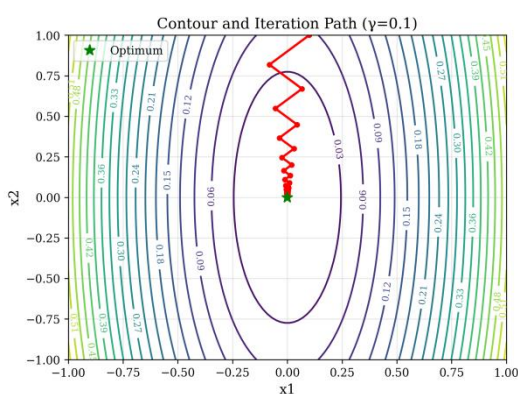
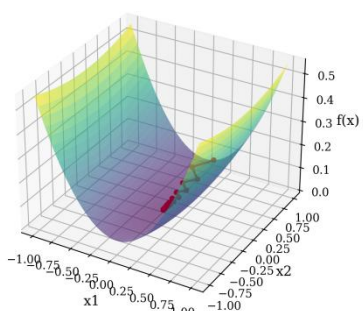


2. 三维曲面与二维等高线轨迹随 γ 值的变化

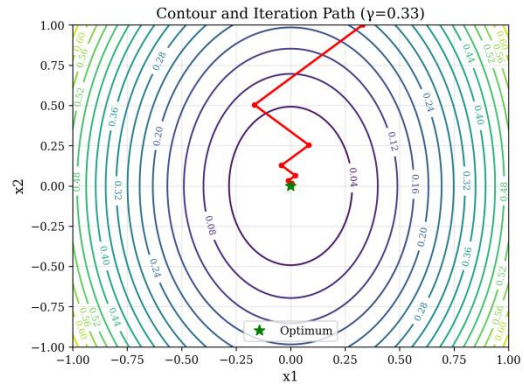
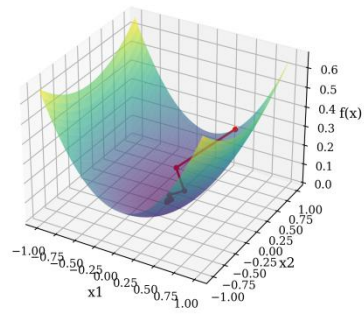
3D Surface and Iteration Trajectory ($\gamma=0.01$)



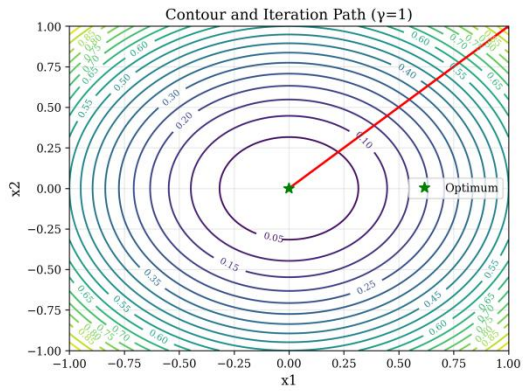
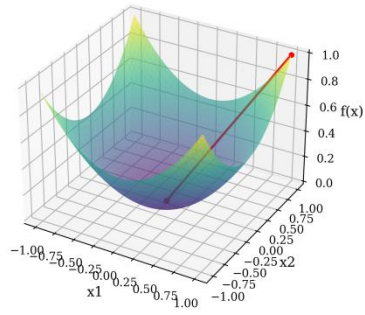
3D Surface and Iteration Trajectory ($\gamma=0.1$)



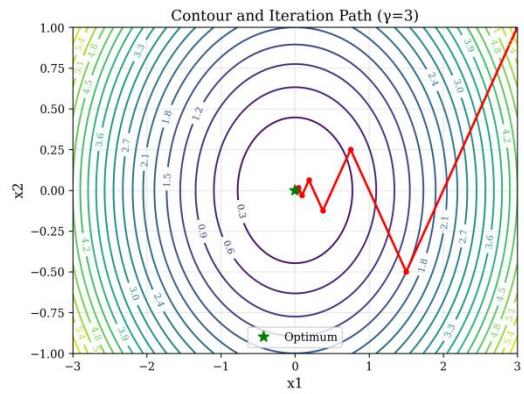
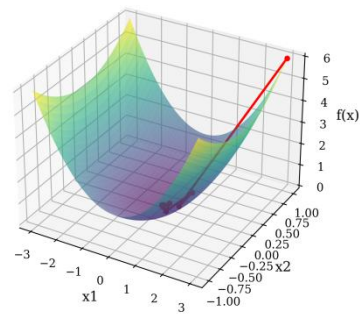
3D Surface and Iteration Trajectory ($\gamma=0.33$)



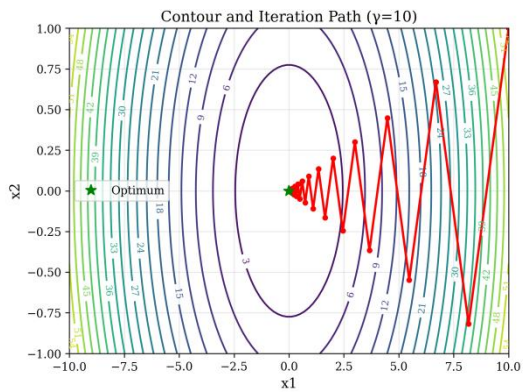
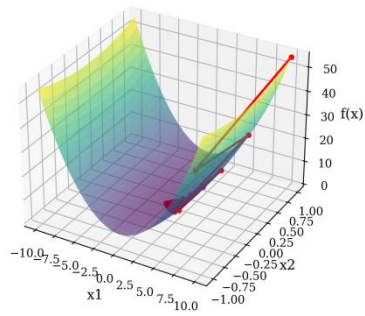
3D Surface and Iteration Trajectory ($\gamma=1$)

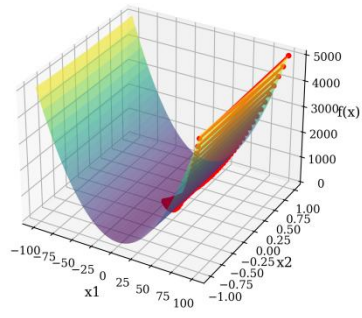
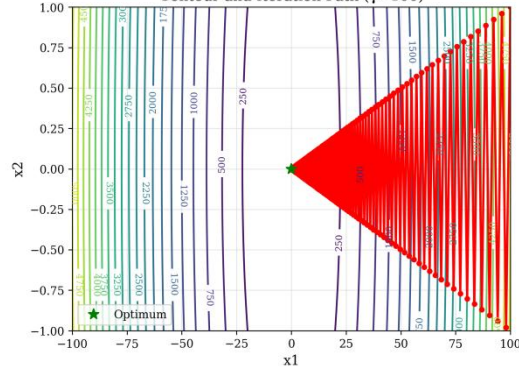


3D Surface and Iteration Trajectory ($\gamma=3$)



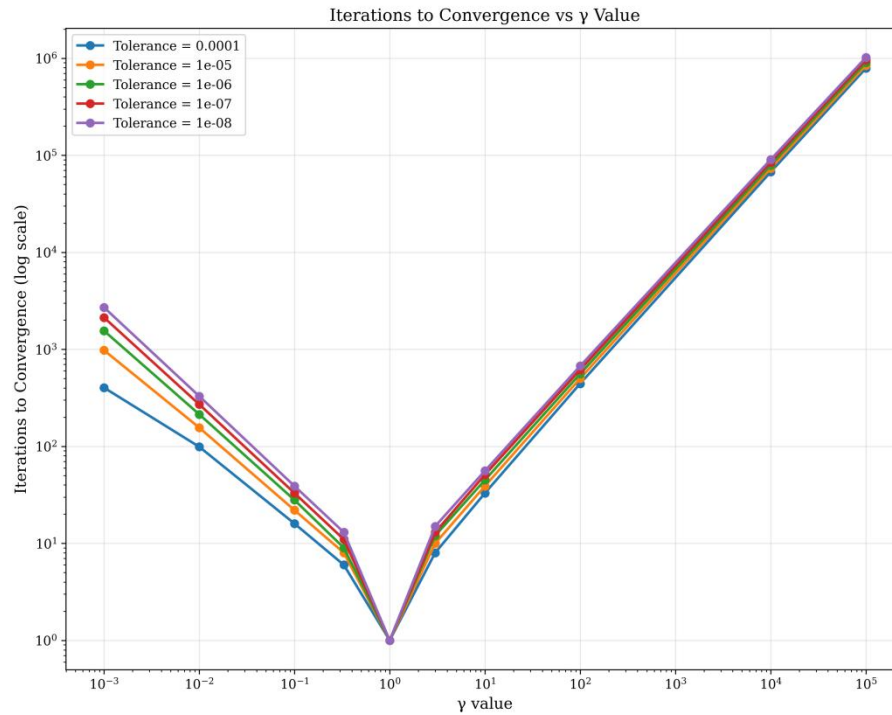
3D Surface and Iteration Trajectory ($\gamma=10$)



3D Surface and Iteration Trajectory ($\gamma=100$)Contour and Iteration Path ($\gamma=100$)

3. 不同 γ 值与容差下的收敛迭代参数

γ	tolerance_values					M/m	Convergence Rate
	1e-4	1e-5	1e-6	1e-7	1e-8		
0.001	403	979	1554	2130	2706	1000	0.998001998
0.01	99	156	214	271	329	100	0.98019802
0.1	16	22	28	33	39	10	0.818181818
0.33	6	8	9	11	13	3.0303	0.503759398
1	1	1	1	1	1	1	0
3	8	10	12	13	15	3	0.5
10	33	39	45	51	56	10	0.818181818
100	444	501	559	617	674	100	0.98019802
10000	67345	73102	78858	84615	90371	10000	0.99980002
100000	788577	846141	903706	961271	1018835	100000	0.99998

不同 γ 值与容差下的收敛迭代次数

六、实验结论

1. $\gamma=1$ 时，一次迭代可得到最优解。
2. 对于 γ 离 1 不远的情况，如 1/3 和 3 之间，收敛速度很快。
3. 如果 $\gamma \gg 1$ 或 $\gamma \ll 1$ 时，收敛速度将会非常慢。

附录：

requirements.txt

```
scipy
pandas
matplotlib
```

run.py

```
import numpy as np
import matplotlib.pyplot as plt
import os
import pandas as pd

if not os.path.exists('result'):
    os.makedirs('result')

y_values_extended = [0.001, 0.01, 0.1, 0.33, 1, 3, 10, 100, 10000, 100000]

def analytical_solution(k, y):
    """Analytical solution:  $x^k = ((y-1)/(y+1))^k * [y, (-1)^k]$ """
    coefficient = ((y - 1) / (y + 1)) ** k
    x1 = coefficient * y
    x2 = coefficient * ((-1) ** k)
    return np.array([x1, x2])

def objective_function(x1, x2, y):
    """Objective function:  $f(x) = 1/2(x1^2 + y*x2^2)$ """
    return 0.5 * (x1**2 + y * x2**2)

def iterations_to_convergence(y_values, tolerance_values, max_iterations=10000000000):
    results = {}
    for y in y_values:
        results[y] = {}
        for tolerance in tolerance_values:
            iterations = 0
            converged = False
            for k in range(max_iterations):
```

```

        x_k = analytical_solution(k, y)
        f_val = objective_function(x_k[0], x_k[1], y)
        if f_val < tolerance:
            iterations = k
            converged = True
            break
    if not converged:
        iterations = max_iterations
    results[y][tolerance] = iterations
df = pd.DataFrame(results).T
df.columns = [f'tol={tol}' for tol in tolerance_values]
df['M/m'] = [max(y, 1/y) for y in y_values]
df['Convergence Rate'] = [abs((y-1)/(y+1)) for y in y_values]
return df

def plot_iterations_to_convergence(y_values, tolerance_values):
    df = iterations_to_convergence(y_values, tolerance_values)
    plt.figure(figsize=(10, 8))
    for i, tolerance in enumerate(tolerance_values):
        col_name = f'tol={tolerance}'
        plt.semilogy(y_values, df[col_name], 'o-', linewidth=2, label=f'Tolerance = {tolerance}')
    plt.xlabel('γ value')
    plt.ylabel('Iterations to Convergence (log scale)')
    plt.title('Iterations to Convergence vs γ Value')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.xscale('log')
    plt.tight_layout()
    plt.savefig('result/iterations_vs_γ.png', dpi=300, bbox_inches='tight')
    plt.show()
    return df

def plot_3d_surface_and_trajectory():
    y_cases = [0.01, 0.1, 0.33, 1, 3, 10, 100]
    for y in y_cases:
        x1_range = max(1, y)
        x1 = np.linspace(-x1_range, x1_range, 100)
        x2 = np.linspace(-1, 1, 100)
        X1, X2 = np.meshgrid(x1, x2)
        Z = objective_function(X1, X2, y)
        max_iterations = 10000
        trajectory_x1 = []
        trajectory_x2 = []
        trajectory_z = []

```



```

for k in range(max_iterations):
    x_k = analytical_solution(k, y)
    z_val = objective_function(x_k[0], x_k[1], y)
    trajectory_x1.append(x_k[0])
    trajectory_x2.append(x_k[1])
    trajectory_z.append(z_val)
fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X1, X2, Z, cmap='viridis', alpha=0.6)
ax1.plot(trajectory_x1, trajectory_x2, trajectory_z, 'ro-', linewidth=2, markersize=4)
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_zlabel('f(x)')
ax1.set_title(f'3D Surface and Iteration Trajectory (  $\gamma = \{y\}$  )')
ax2 = fig.add_subplot(122)
contour = ax2.contour(X1, X2, Z, levels=20)
ax2.clabel(contour, inline=True, fontsize=8)
ax2.plot(trajectory_x1, trajectory_x2, 'ro-', linewidth=2, markersize=4)
ax2.plot(0, 0, 'g*', markersize=10, label='Optimum')
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_title(f'Contour and Iteration Path (  $\gamma = \{y\}$  )')
ax2.legend()
ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(f'result/3d_surface_trajectory_  $\gamma = \{y\}$  .png', dpi=300, bbox_inches='tight')
plt.show()

```

```

def plot_initial_iterations():
    """Plot first few iterations for different y values with publication quality"""
    # Set publication quality style
    plt.rcParams.update({
        'font.family': 'serif',
        'font.size': 11,
        'axes.labelsize': 12,
        'axes.titlesize': 13,
        'legend.fontsize': 10,
        'xtick.labelsize': 10,
        'ytick.labelsize': 10,
        'lines.linewidth': 1.5,
        'lines.markersize': 6,
    })
    y_values = [0.1, 0.5, 1, 2, 3, 5]
    colors = plt.cm.Set2(np.linspace(0, 1, len(y_values)))

```

```

markers = ['o', 's', '^', 'D', 'v', '<']
max_iterations = 10
fig, ax = plt.subplots(figsize=(6, 4.5))
for i, y in enumerate(y_values):
    f_values = []
    for k in range(max_iterations):
        x_k = analytical_solution(k, y)
        f_val = objective_function(x_k[0], x_k[1], y)
        f_values.append(f_val)
    ax.plot(range(max_iterations), f_values,
            color=colors[i],
            marker=markers[i],
            linewidth=1.5,
            label=f'y = {y} (M/m = {max(y, 1/y):.1f})')
ax.set_xlabel('Iteration $k$')
ax.set_ylabel('Objective Value $f(x^k)$')
ax.set_title('First few iterations for Different Condition Numbers', pad=15)
ax.legend(loc='upper right', frameon=True, framealpha=0.9)
ax.grid(True, alpha=0.3)
ax.set_xlim(-0.5, max_iterations - 0.5)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.savefig('result/initial_iterations.png', bbox_inches='tight', dpi=300)
plt.show()

print("Generating visualizations...")
print("First few iterations")
plot_initial_iterations()
print("3D surface and trajectory")
plot_3d_surface_and_trajectory()
print("Iterations to convergence analysis")
tolerance_values = [0.0001, 0.00001, 0.000001, 0.0000001, 0.00000001]
df = plot_iterations_to_convergence(y_values_extended, tolerance_values)
print("\nIterations to convergence for different  $\gamma$  values and tolerances:")
print(df)
df.to_csv('result/iterations_to_convergence_table.csv')
print("\nIterations table saved to 'result/iterations_to_convergence_table.csv'")

```